

Reviewer Pack — Additive Energy Threshold for ACC⁰ Circuit Size in CNF Formu...

Entry 0553f6013ae8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 00:43:12 UTC

Additive Energy Threshold for ACC⁰ Circuit Size in CNF Formulas

Entry ID: 0553f6013ae8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 00:43:12 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics **Field B** (complexity object): ACC⁰ Circuit Size for CNF Formulas

Statement:

For any CNF formula Φ with n variables, if the additive energy of the set of satisfying assignments S is $\Omega(n^3)$, then Φ cannot be computed by an ACC⁰ circuit of size $o(n^2)$.

Rationale (proposer's reasoning):

Additive energy measures the structure of the solution set; ACC⁰ circuits have limited capacity to capture such complex structures, leading to a lower bound on circuit size.

Taxonomy category: ACC_SIPSER (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6d5e7956419b6f10

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from itertools import combinations

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

def generate_primes(count):
    primes = []
    num = 2
    while len(primes) < count:
        if is_prime(num):
            primes.append(num)
        num += 1
    return primes

def random_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = [random.choice([-i, i]) for i in range(1, n + 1)]
        cnf.append(clause)
    return cnf

def evaluate_cnf(cnf, assignment):
    for clause in cnf:
        if not any([assignment[abs(lit) - 1] == (lit > 0) for lit in clause]):
            return False
    return True

def add_energy(cnf, assignment):
    count = 0
    for x, y, z, w in combinations(range(n), 4):
        if evaluate_cnf(cnf, [assignment[x], assignment[y], assignment[z], assignment[w]]):
            count += 1
    return count

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.choice([5, 10, 15, 20, 30, 40])
    m = 2 * n
```

```

cnf = random_cnf(n, m)
assignment = [random.choice([0, 1]) for _ in range(n)]

energy = add_energy(cnf, assignment)
threshold = 0.8 * n**3
circuit_size = 0.5 * n**2

conjecture_holds = energy < threshold or len(cnf) >= circuit_size
counterexample = "" if conjecture_holds else f"Energy={energy}, Circuit size={len(cnf)}"

return {
    "metric_name": "Additive Energy",
    "metric_value": energy,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or generate_primes(30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_energy = sum(r["metric_value"] for r in results) / len(results)
    std_energy = (sum((r["metric_value"] - mean_energy)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_energy} std={std_energy} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_89dca52f.py", line 83, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_89dca52f.py", line 63, in run_trial

```

    energy = add_energy(cnf, assignment)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_89dca52f.py", line 49, in add_energy

```
for x, y, z, w in combinations(range(n), 4):
    ^
```

NameError: name 'n' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined variable 'n' in the add_energy function, preventing data collection. | next: Fix the NameError in test_89dca52f.py by defining 'n' before using it in combinations

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	80089
2	preregistration	ollama_remote	qwen3:8b	0	0	24242
3	novelty	ollama_remote	qwen3:8b	0	0	20686
4	novelty	ollama_remote	qwen3:8b	0	0	13931
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13981
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8761
7	judge	ollama_remote	qwen3:8b	0	0	17942

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 179631 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/0553f6013ae8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0553f6013ae8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0553f6013ae8.tar.gz` (if generated)